

Bases de datos

UD3.Diseño físico de la base de datos



ÍNDICE

Introducción

El lenguaje SQL

Lenguaje DDL: Data Definition Language

Tipos de Datos.

Conversión de Datos.

Expresiones y operadores condicionales

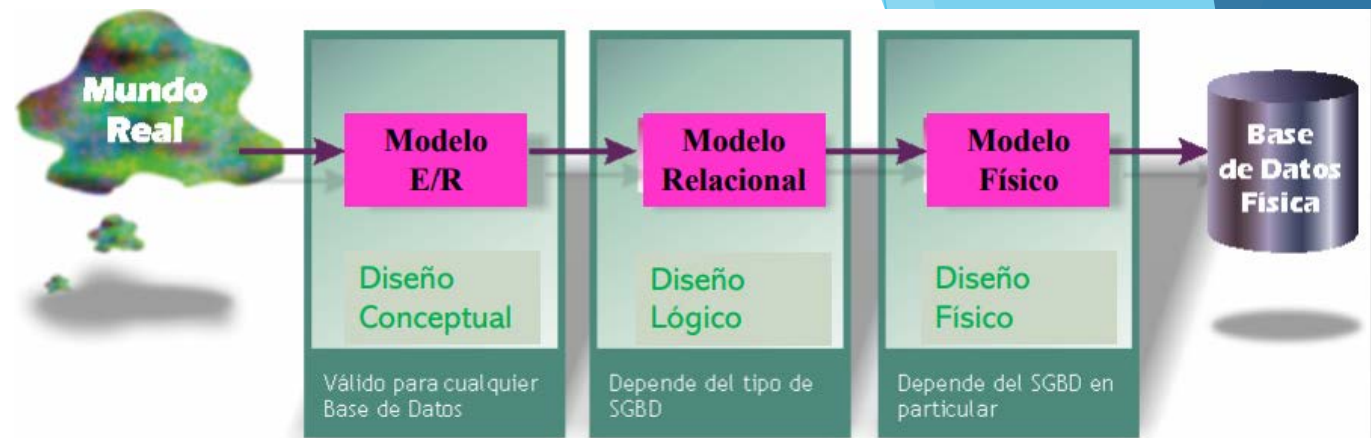
A1: DISEÑAR LAS RELACIONES BASE PARA EL SGBD ESPECÍFICO

Creación, Modificación y Eliminación de BBDD.

Creación, Modificación y Eliminación de Tablas.

A2: DISEÑAR LAS REGLAS DE NEGOCIO PARA EL SGBD ESPECÍFICO

Introducción



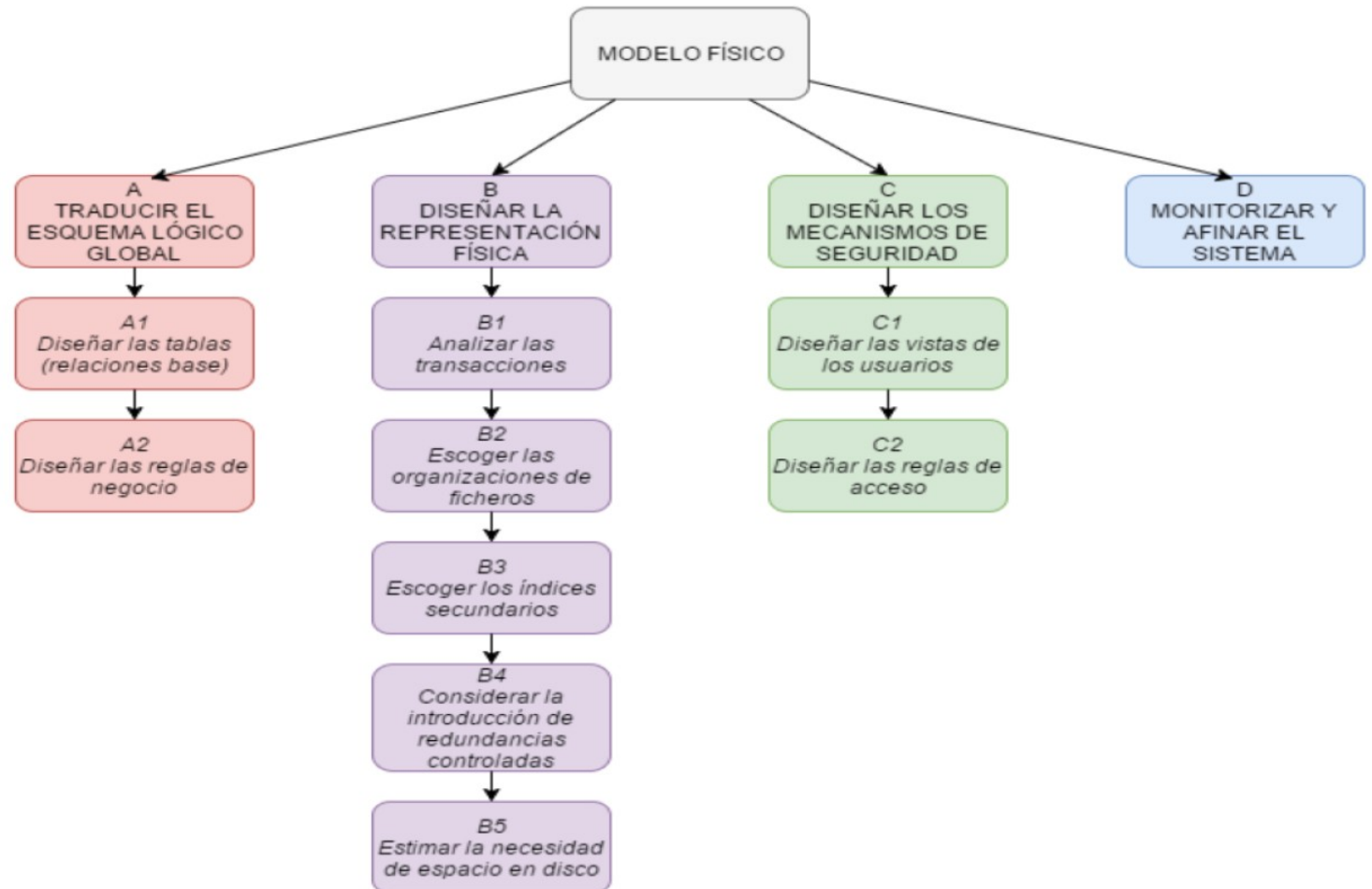
- ▶ Como ya hemos visto anteriormente, el diseño de las bases de datos pasa por las siguientes **fases**:
 - ▶ FASE 1. Diseño Conceptual - Modelo Entidad/Relación (E/R). Consiste en representar la información obtenida del usuario final, y concretada en la especificación de los requisitos, mediante estándares para que el resto de la comunidad informática pueda entender y comprender el modelo deseado.
 - ▶ FASE 2. Diseño Lógico - Modelo Relacional. Este modelo es más técnico que el anterior porque está orientado al personal informático y generalmente tiene traducción directa al modelo físico que entiende el SGBD. Dependerá del tipo de la BD (jerárquica, orientada a objetos, relacional, ...)
 - ▶ FASE 3. Es el resultado de aplicar el modelo lógico a un SGBD concreto. Generalmente está expresado en un lenguaje de programación de BBDD tipo SQL. El Modelo Relacional se transformará en el modelo físico mediante el sublenguaje DDL de SQL.

En este tema vamos a aprender a realizar el diseño físico de la base de datos.

Introducción

El diseño físico de BBDD tiene las fases descritas en la imagen.

En este tema nos centraremos en la fase A, principalmente.



Introducción

A: TRADUCIR EL ESQUEMA LÓGICO GLOBAL

- ▶ Traducir el esquema lógico global en un esquema que se pueda implementar en el SGBD escogido.
- ▶ Es necesario conocer toda la funcionalidad que este SGBD ofrece.
- ▶ El diseñador deberá saber:
 - ▶ Si el sistema soporta la definición de claves primarias, claves ajenas y claves alternativas.
 - ▶ Si el sistema soporta la definición de datos requeridos (es decir, si se pueden definir atributos como no nulos).
 - ▶ Si el sistema soporta la definición de dominios.
 - ▶ Si el sistema soporta la definición de reglas de negocio.
 - ▶ Cómo se crean las tablas base.

El lenguaje SQL

CARACTERÍSTICAS

- ▶ En SQL no se distingue entre mayúsculas y minúsculas.
- ▶ El final de una instrucción o sentencia lo marca el signo de punto y coma (;).
- ▶ Las sentencias SQL se pueden escribir en varias líneas siempre que las palabras no se dividan.
- ▶ Los comentarios se pueden escribir de dos formas:

- ▶ Para comentar un bloque de líneas: `/*` `*/`

Ej:

```
/*  
    línea 1  
    línea 2  
*/
```

- ▶ Para comentar una línea: `--`

Ej:

```
-- línea 1
```

NOTACIÓN.

- Las palabras clave se suelen escribir en mayúsculas.
- Los corchetes `[ ]` expresan opcionalidad.
- Las llaves `{ op1|op2 }` expresan alternativas de las cuales se ha de elegir una.
- Los puntos suspensivos `...` indican la repetición de la opción anterior.

El lenguaje SQL

► Una instrucción SQL se ejecuta de la siguiente forma:

1. Se analiza y se comprueba que la sintaxis es correcta. (Palabras reservadas,...)
2. Si es correcta, usa el diccionario de datos para verificar que los metadatos sean correctos también. (si existen los campos, la tabla, si el tipo es correcto,...)
3. Si son correctos, se realiza un pequeño proceso de optimización, con el fin de consumir los mínimos recursos posibles.
4. Por último, se ejecuta la sentencia y se muestra el resultado de la misma.

El lenguaje SQL

Comandos útiles

- ▶ Este comando muestra la lista de usuarios:

```
SELECT user FROM mysql.user;
```

- ▶ Este comando muestra todas las tablas:

```
SHOW TABLES [FROM database];
```

- ▶ Este comando muestra las columnas de una tabla:

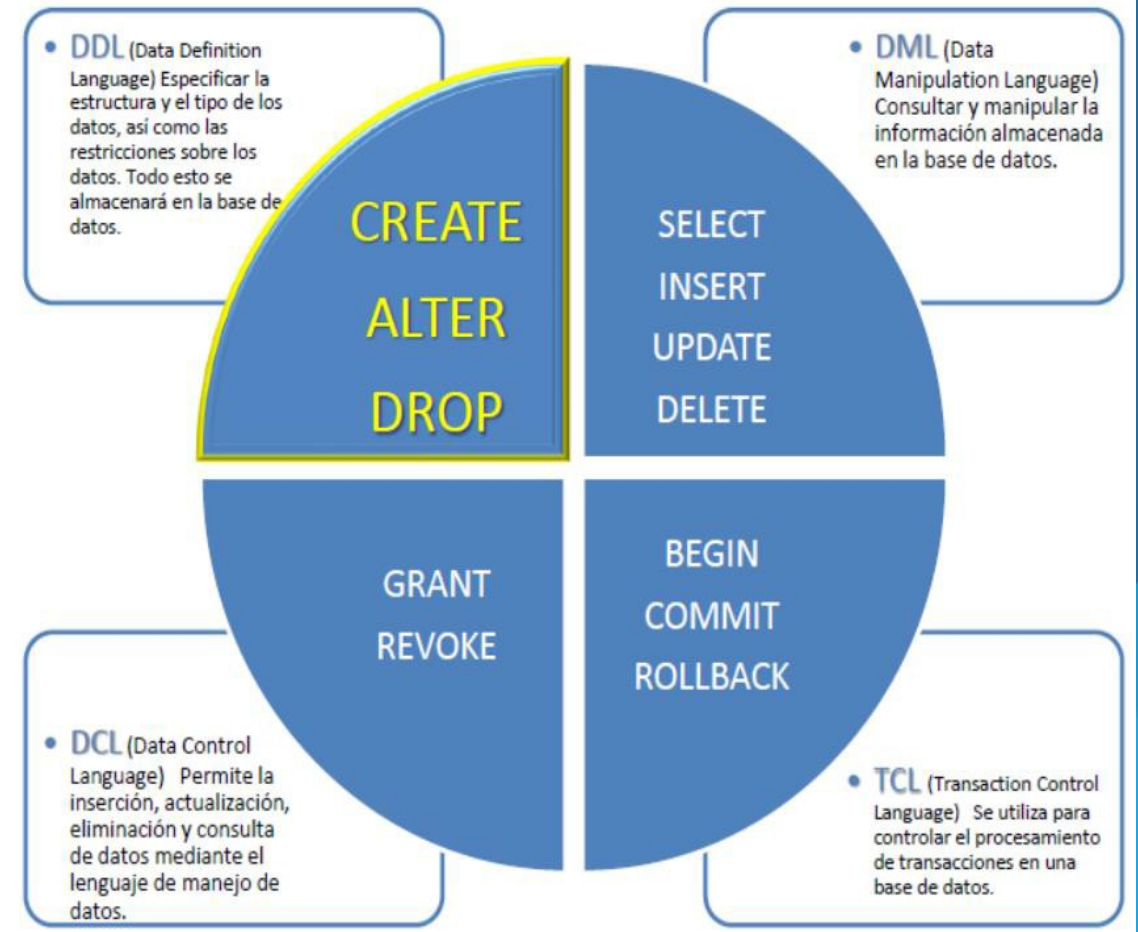
```
SHOW COLUMNS FROM database.name;
```

También podemos usar DESC o DESCRIBE:

```
DESCRIBE database.name;
```

El lenguaje SQL

- ▶ Los elementos de este lenguaje, se agrupan en tres subconjuntos:
 - ▶ **DDL (Data Definition Language)**. Es el lenguaje de definición de datos y nos permite definir, modificar o eliminar los distintos objetos de la base de datos (tablas, vistas, índices...).
 - ▶ **DML (Data Manipulation Language)**. Es el lenguaje de manipulación de datos y nos permite insertar, suprimir y modificar registros o filas de las tablas. También nos permite la realización de consultas sobre la BD (SELECT).
 - ▶ **DCL (Data Control Language)**. Es el lenguaje de control de datos. Nos permite establecer derechos de acceso de los usuarios sobre los distintos objetos de la base de datos. Lo forman las instrucciones GRANT y REVOKE.
- ▶ Hay otro tipo de sentencias para realizar transacciones, que gestionan las modificaciones creadas por las instrucciones DML. Lo forman las instrucciones ROLLBACK, COMMIT y SAVEPOINT.



DDL (Data Definition Language).

- ▶ El lenguaje de definición de datos (Data Definition Language, DDL) es un lenguaje proporcionado por un sistema gestor de base de datos que permite a los programadores del mismo llevar a cabo las tareas de definición de las estructuras que almacenarán los datos.
- ▶ Aunque SQL es un estándar cada SGBD realiza una implementación y adaptación propia. DDL es parte de SQL y, por tanto, ocurre lo mismo.
- ▶ ¿Qué operaciones vamos a hacer?
 - ▶ CREATE : Creación
 - ▶ ALTER : Modificación
 - ▶ DROP : Borrado
- ▶ ¿Sobre qué?
 - ▶ Bases de datos
 - ▶ Tablas

DDL (Data Definition Language).

Tipos de Datos

CHAR(n)	Alfanumérico de longitud fija. El número de caracteres que tendrá es (n). Rellena con espacios en blanco <i>Ej: CHAR(2) → '2A'</i>
VARCHAR(n)	Alfanumérico de longitud variable. El número de caracteres máximo que tendrá es (n). <i>Ejs: VARCHAR(4) → '2A', '3R6', '9AS1'</i>
INT(n) INTEGER(N)	Numérico de un máximo de 40 dígitos. Se usa para Números enteros. El número de dígitos es (n). <i>Ej: INT(10) → 1000000000</i>
DECIMAL(p,d)	Numérico para definir números reales. El número total de dígitos es (p) y el número total de decimales es (d). <i>Ej: DECIMAL(4,2) → 33,45 (el máximo número representable sería 99.99)</i>
DATE	Permite almacenar fechas. Tiene numerosas opciones de formato. Por defecto: 'YYYY-MM-DD'
DATETIME	Permite almacenar datos con fecha y hora. Por defecto: 'YYYY-MM-DD HH:MM:SS'
BOOL, BOOLEAN	Permite los valores 'TRUE' y 'FALSE'

DDL (Data Definition Language).

Tipos de Datos

Cadenas de caracteres: CHAR y VARCHAR

CHAR(n)	Alfanumérico de longitud fija. El número de caracteres que tendrá es (n). Rellena con espacios en blanco <i>Ej: CHAR(2) → '2A'</i>
VARCHAR(n)	Alfanumérico de longitud variable. El número de caracteres máximo que tendrá es (n). <i>Ejs: VARCHAR(4) → '2A', '3R6', '9AS1'</i>

- ▶ Se escriben entre comillas simples. Ej: 'lenguaje SQL'
- ▶ Cuando usamos cadenas variable, hay que intentar no quedarse corto. El espacio que no se usa no se va a desaprovechar.

DDL (Data Definition Language).

Tipos de Datos

- ▶ Operadores más frecuentes:

Operador	Tipo resultado	Ejemplo
Igual(=)	Verdadero o falso	'hola'='hola' → Verdad (TRUE)
Distinto (!=)(<>)	Verdadero o falso	'hola'<>'hola' → Falso (FALSE)
CONCAT()	Cadena	CONCAT('Hola',' mundo') → 'Hola mundo'
Longitud (Length(c))	Numérico	Length('mundo') → 5
Minúsculas (LOWER(c))	Cadena	Lower('Hola') → 'hola'
Mayúsculas(UPPER(c))	Cadena	Upper('Hola') → 'HOLA'

- ▶ Otros operadores útiles: INSTR(c, subc), LTRIM(c), RTRIM(c), SUBSTR(c, pos, len)

DDL (Data Definition Language).

Tipos de Datos

Números: INT(n), DECIMAL(p,d)

INT(n)
INTEGER(N) Numérico de un máximo de 40 dígitos. Se usa para Números enteros. El número de dígitos es (n).

Ej: INT(10) → 1000000000

DECIMAL(p,d) Numérico para definir números reales. El número total de dígitos es (p) y el número total de decimales es (d).

Ej: DECIMAL(4,2) → 33,45 (el máximo número representable sería 99.99)

Operadores más frecuentes:

Operador	Tipo resultado	Ejemplo
+ , - , * , / , MOD	Número	4+4→8, 4-4→0, 4*4→16, 4/4→1
ABS(n)	Número	ABS(-43) = 43
SQRT(n)	Número	SQRT(144) = 12
POWER(b,e)	Número	POWER(2,8) = 256

DDL (Data Definition Language).

Tipos de Datos

Fechas: DATE

DATE	Permite almacenar fechas. Tiene numerosas opciones de <u>formato</u> . Por defecto: 'YYYY-MM-DD'
DATETIME	Permite almacenar datos con fecha y hora. Por defecto: 'YYYY-MM-DD HH:MM:SS'

- ▶ Las fechas se pueden escribir en numerosos formatos. Se escriben entre comillas simples y el separador es el guión (-) .
- ▶ Operadores más frecuentes:
 - SYSDATE()**: función que proporciona la fecha actual (fecha y hora). Función similar: NOW().
 - CURDATE()**: función que devuelve la fecha de hoy en formato solo fecha, sin la hora.

DDL (Data Definition Language).

Conversión de Datos

- ▶ **Implícita.** El SGBD la hace automáticamente. Cuando el SGBD encuentra una expresión, por ejemplo, un dato de un tipo diferente al esperado, aplica una serie de reglas para intentar convertir ese dato al tipo esperado.

Ej: Intento introducir en un NUMBER la cadena '55', el SGBD puede traducirlo y almacenar el valor 55.

- ▶ **Explícita.** Se realiza mediante operadores de conversión. Algunas de estas funciones son:
 - ▶ Conversión cadena-fecha: STR_TO_DATE()
 - ▶ CONVERT(value, type)

DDL (Data Definition Language).

Expresiones y operadores condicionales

- ▶ Las condiciones son expresiones lógicas (devuelven verdadero o falso). Dentro del DDL se utilizarán con la cláusula CHECK que sirve para establecer las condiciones que deben cumplir sobre los valores que se almacenarán en una tabla.

- ▶ Operadores de **comparación**: =, <>, !=, <=, >=, < , >.

Ejs: horas >= 10.5 , nombre = 'PEPE', fecha < '1-ene-93'

- ▶ **[NOT] IN** lista. Comprueba que un valor está o no en una lista de valores (que puede ser el resultado de una consulta).

Ej: nombre NOT IN ('DANI', 'LIDIA')

- ▶ **oper {ANY|SOME}** lista. Comprueba que se cumple la operación de comparación *oper* con algún elemento de la lista.

Ej: nombre = ANY ('DANI', 'LIDIA')

- ▶ **oper ALL** lista. Comprueba que se cumple la operación de comparación *oper* con todos los elementos de la lista.

Ej: nombre <> ALL ('DANI', 'LIDIA')

DDL (Data Definition Language).

Expresiones y operadores condicionales

- ▶ **[NOT] BETWEEN x AND y.** Comprueba si el valor pertenece al rango x - y.
Ej: minutos NOT BETWEEN 0 and 60
- ▶ **[NOT] EXISTS lista.** Comprueba si la lista de valores contiene algún elemento.
*Ejs: EXISTS ('1', '2') devuelve verdadero (TRUE).
 NOT EXISTS ('1', '2') devuelve falso (FALSE).*
- ▶ **[NOT] LIKE texto.** Permite comparar cadenas alfanuméricas haciendo uso de símbolos comodín.
 - ▶ *_ : sustituye a un único carácter.*
 - ▶ *% : sustituye a varios caracteres.**Ejs: nombre LIKE 'Luis%'
 codigo NOT LIKE '2_'*
- ▶ **IS [NOT] NULL.** Comprueba si el campo está vacío y devuelva verdadero (TRUE) o falso (FALSE).
Ej: nombre IS NOT NULL

DDL (Data Definition Language).

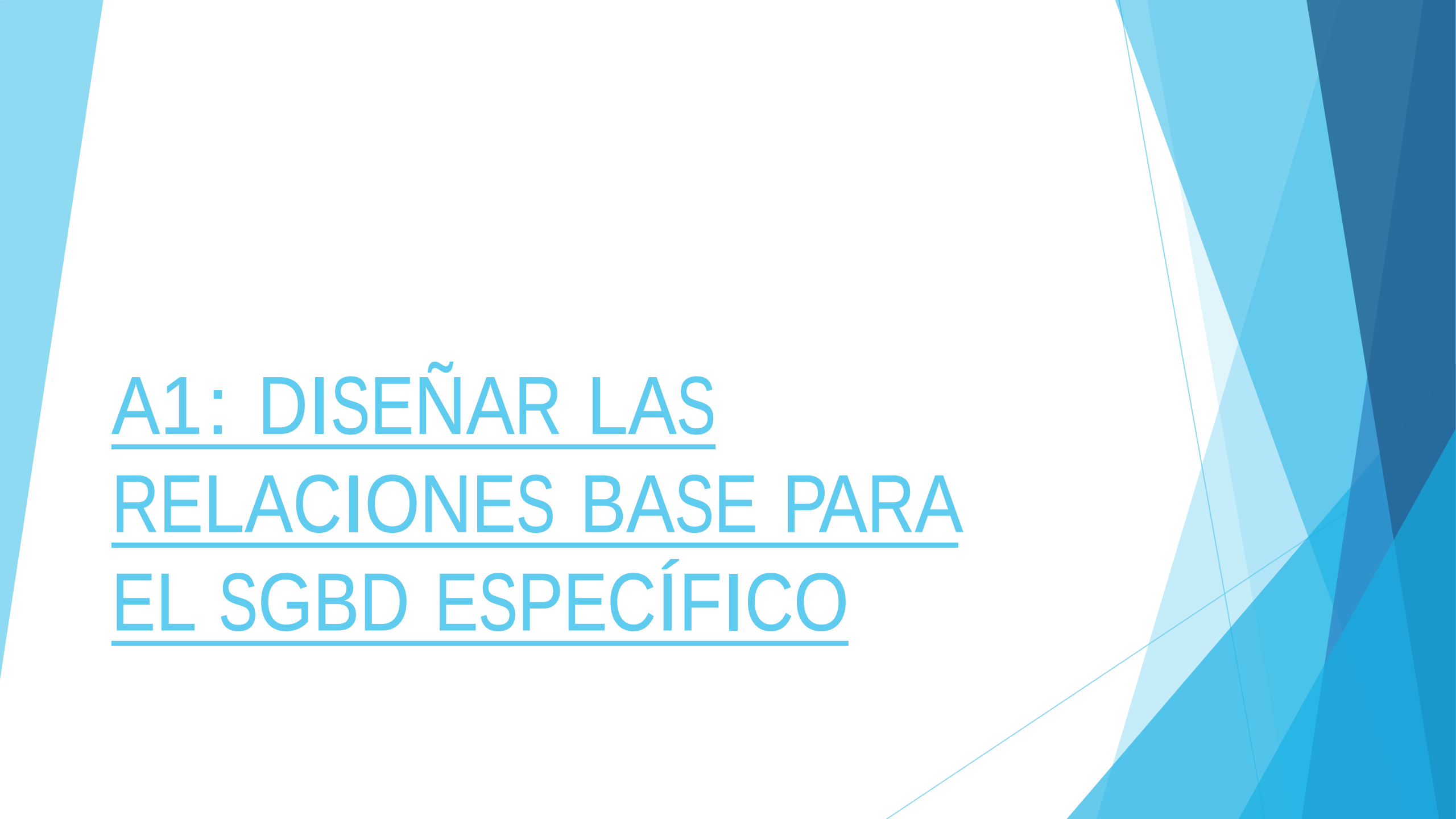
Expresiones y operadores condicionales

- ▶ Operadores lógicos: **OR**, **AND**, **NOT**.
- ▶ Los operadores lógicos junto con el uso de paréntesis permiten combinar condiciones simples obteniendo otras más complejas. Los operadores lógicos son:
 - ▶ condición1 **OR** condición2: Cierto siempre excepto cuando las 2 condiciones sean falsas.
 - ▶ condición1 **AND** condición2: Falso siempre excepto cuando las 2 condiciones sean ciertas.
 - ▶ **NOT** condición: Invierte la condición.

Ejs: (1=1) AND (1=2) → FALSE

(1<2) OR (1=0) → TRUE

NOT (1=1) → FALSE

The background features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue. These shapes are primarily located on the right side and bottom of the frame, creating a modern, dynamic feel.

A1: DISEÑAR LAS RELACIONES BASE PARA EL SGBD ESPECÍFICO

Diseño de relaciones

- ▶ Las relaciones base se definen mediante el lenguaje de definición de datos del SGBD.
- ▶ Para ello, se utiliza la información producida durante el diseño lógico: el esquema lógico global y el diccionario de datos.
- ▶ El esquema lógico consta de un conjunto de relaciones (tablas) y, para cada una de ellas, se tiene:
 - ▶ El nombre de la relación.
 - ▶ La lista de atributos.
 - ▶ La clave primaria y las claves foráneas, si las tiene.
 - ▶ Las reglas de integridad de las claves foráneas (no representadas en el diagrama).

Diseño de relaciones

- ▶ En el diccionario de datos se describen los atributos y, para cada uno de ellos, se tiene:
 - ▶ Su dominio: tipo de datos, longitud y restricciones de dominio.
 - ▶ El valor por defecto, que es opcional.
 - ▶ Si admite nulos.
 - ▶ Si es derivado y, en caso de serlo, cómo se calcula su valor.
- ▶ Para diseñar las relaciones base (tablas) se utiliza el lenguaje SQL (Structured Query Language). Hemos hablado del lenguaje SQL. Es el lenguaje más extendido para la definición y manipulación de datos en SGBDR.

Creación, Modificación, Uso y Eliminación de BBDD

Creación de una Base de datos

- ▶ El comando SQL de creación de una base de datos es **CREATE DATABASE**, seguido del nombre que se elija. Se generará un esquema que nos permitirá organizar las tablas dentro de nuestra base de datos.
 - ▶ Ej: *CREATE DATABASE Ejercicio_1;*

Eliminación de una Base de datos

- ▶ La sentencia que se utiliza para ello es **DROP DATABASE**.
 - ▶ Ej: *DROP DATABASE Ejercicio_1;*

Modificación de una Base de datos

- ▶ Se utiliza la sentencia **ALTER DATABASE** que posee muchísimas cláusulas, dependiendo del parámetro que se desee modificar.

Uso de una Base de datos.

- ▶ Una vez creada una base de datos, para acceder usaremos la instrucción **USE** seguido del nombre de la BD.
 - ▶ Ej: *USE Ejercicio_1;*

Creación de tablas.

Características del nombre.

Cuando se crea una tabla, el nombre de la misma ha de cumplir las siguientes condiciones:

- ▶ Deben comenzar con una letra
- ▶ Longitud máxima:30 caracteres
- ▶ Intentar no usar caracteres especiales (se recomienda no usar \$ y # especialmente)
- ▶ No pueden existir dos tablas con el mismo nombre para el mismo usuario (dentro del mismo esquema)
- ▶ No se pueden usar palabras reservadas de SQL como nombre

Creación de tablas.

- ▶ Para la **creación** de tablas, se usa la sentencia **CREATE TABLE**. La sintaxis básica de este comando es:

```
CREATE TABLE nombre_tabla (  
  columna1 tipo_dato [ restricciones de columna1 ],  
  columna2 tipo_dato [ restricciones de columna2 ],  
  columna3 tipo_dato [ restricciones de columna3 ],  
  ...  
  [ restricciones de tabla ]  
);
```

- ▶ Ejemplo:

```
CREATE TABLE usuarios (  
  id          INT(5) PRIMARY KEY,  
  Dni         CHAR(9) UNIQUE,  
  Nombre      VARCHAR(50) NOT NULL,  
  edad        INT(3) CHECK (edad>0 and edad<120)  
);
```

Creación de tablas. Restricciones.

PRIMARY KEY	Define el/los atributos que forman la clave primaria. Ya recoge las restricciones UNIQUE y NOT NULL.
UNIQUE	Impide que se introduzcan valores repetidos para un atributo/conjunto de atributos. No se puede utilizar junto con PRIMARY KEY. Se utiliza para claves alternativas.
NOT NULL	Impide que se introduzcan filas en la tabla con valor NULL para ese atributo. No se utiliza con PRIMARY KEY.
DEFAULT	Permite asignar un valor por defecto al campo que se está definiendo.
CHECK	Permite establecer condiciones que deben cumplir los valores de dicha columna. Reglas: • Se puede poner a nivel de columna o a nivel de tabla. • Sólo permite condiciones simples (ni referencias a otra tabla ni consultas) • Permite usar NOT, AND, OR, pero no permite usar SYSDATE ni USER.

Creación de tablas.

Restricciones a nivel de COLUMNA.

- ▶ Las restricciones de cada **COLUMNA** de la tabla, tienen la sintaxis:

```
CONSTRAINT nombre_restricción {  
  [NOT] NULL | UNIQUE | PRIMARY KEY | DEFAULT valor | CHECK (condición)  
}
```

- ▶ Se pueden poner directamente después del tipo de la columna, o después de la definición de tipos usando la instrucción anterior. La palabra CONSTRAINT es *opcional* también. Si la usamos, podremos ponerle un nombre a la restricción, si no, el SGBD asignará el nombre a la restricción.
- ▶ Ejemplo: estas tablas son iguales

```
CREATE TABLE CONTACTOS_v1(  
  cod INT NOT NULL PRIMARY KEY ,  
  nombre VARCHAR(30) NOT NULL,  
  edad INT NOT NULL CHECK (edad>18),  
  ciudad VARCHAR(30)  
);
```

```
CREATE TABLE CONTACTOS_v2(  
  cod INT,  
  nombre VARCHAR(30) NOT NULL,  
  edad INT NOT NULL,  
  ciudad VARCHAR(30),  
  PRIMARY KEY(cod),  
  CONSTRAINT con_ck CHECK (edad>18)  
);
```

Creación de tablas.

Restricciones a nivel de TABLA.

- ▶ Las restricciones a nivel de **TABLA** se definen cuando una misma restricción afecta a varias columnas y tienen la sintaxis:

```
CONSTRAINT nombre_restricción {  
    PRIMARY KEY (columna1 [,columna2] ... )  
| UNIQUE (columna1 [,columna2] ... )  
| FOREIGN KEY (columna1 [,columna2] ... )  
    REFERENCES nombre_tabla (columna1 [,columna2] ... )  
    [ON DELETE {CASCADE | SET NULL}]  
| CHECK (condición)  
}
```

```
create table vehiculos(  
    patente CHAR(6) NOT NULL,  
    tipo CHAR(4),  
    horallegada TIME NOT NULL,  
    horasalida TIME,  
    CONSTRAINT PRIMARY KEY(patente,horallegada),  
    CHECK (horallegada >= horasalida)  
);
```

Creación de tablas.

Creación de Foreign keys.

- ▶ Vemos que a nivel de **TABLA** tenemos una restricción nueva, la de FOREIGN KEY. Mediante esta restricción asociamos dos tablas que tienen una relación en la que una referencia a la otra.

FOREIGN KEY	Define claves externas que hacen referencia a otras tablas. Debe referenciar a una clave primaria completa de la tabla padre, y nunca a un subconjunto de los atributos que forman esta clave primaria. Para poder definirla, la tabla padre debe estar creada
-------------	--

- ▶ Para poder crear una restricción de este tipo en el momento de la creación de una tabla, debemos asegurarnos de que la tabla a la que hacer referencia esté creada previamente en nuestra base de datos. Así, en el siguiente ejemplo, para poder crear la tabla **COCHES**, deberíamos crear previamente **taller** y **propietario**.

```
CREATE TABLE COCHES (  
  Matricula VARCHAR(8),  
  marca VARCHAR(15),  
  color VARCHAR(15),  
  codTaller VARCHAR(10),  
  codProp VARCHAR(10),  
  PRIMARY KEY (matricula),  
  FOREIGN KEY (codTaller) REFERENCES taller (codTaller),  
  FOREIGN KEY (codProp) REFERENCES propietario (codProp),  
  CHECK (color IN ('ROJO','AZUL','BLANCO','GRIS','VERDE','NEGRO'))  
);
```

```
CREATE TABLE propietario (  
  codProp varchar(10) NOT NULL,  
  PRIMARY KEY (codProp)  
);
```

```
CREATE TABLE taller (  
  codTaller varchar(10) NOT NULL,  
  PRIMARY KEY (codTaller)  
);
```

Borrado de tablas.

- ▶ Para el borrado de tablas, se usa la sentencia **DROP TABLE**. La sintaxis básica de este comando es:

```
DROP TABLE nombre_tabla  
[ CASCADE CONSTRAINTS ];
```

- ▶ La opción CASCADE CONSTRAINTS permite eliminar una tabla que contenga atributos referenciados por otras tablas, eliminando también todas esas referencias.
- ▶ Si la clave principal de la tabla es una clave externa en otra tabla y no se utiliza la opción CASCADE CONSTRAINTS, entonces no se podrá eliminar la tabla.
- ▶ **El borrado de una tabla es irreversible y no hay ninguna petición de confirmación, por lo que conviene ser muy cuidadoso con esta operación. Al borrar una tabla se borran todos los datos que contiene.**

Modificación de tablas.

- ▶ Cambiar el nombre de una tabla.

```
RENAME TABLE nombre_actual TO nombre_nuevo;
```

Ej:

```
RENAME TABLE COCHES TO VEHICULOS;
```

- ▶ Borrar TODO el contenido de una tabla.

```
TRUNCATE TABLE nombre_tabla;
```

La orden TRUNCATE TABLE seguida del nombre de una tabla, hace que se elimine el contenido de la tabla, pero no la tabla en sí. Incluso borra del archivo de datos el espacio ocupado por la tabla. (Esta orden no puede anularse con un ROLLBACK)

Alteración de tablas. Cambios estructurales.

ADD.

Para hacer cambios estructurales en tablas, se usa **ALTER TABLE** con diferentes opciones.

- ▶ **Añadir columnas:** **ADD** permite añadir columnas.

```
ALTER TABLE nombre ADD (  
  columna1 tipo [ restricciones ],  
  columna2 tipo [ restricciones ]  
  ... ]  
);
```

```
ALTER TABLE nombre  
ADD COLUMN  columna1 tipo [ restricciones ],  
ADD COLUMN  columna2 tipo [ restricciones ],  
... ]  
);
```

- ▶ *Ej:*

```
ALTER TABLE VEHICULOS ADD (  
  fechaMatric      DATE,  
  tipoFaros        VARCHAR(20) NOT NULL  
);
```

```
ALTER TABLE propietario  
ADD COLUMN nombre VARCHAR(25) NOT NULL,  
ADD COLUMN DNI VARCHAR(10) NOT NULL  
;
```

Alteración de tablas. Cambios estructurales.

MODIFY.

- **Modificar columnas:** **MODIFY** permite cambiar el tipo y propiedades de las columnas.

```
ALTER TABLE nombre_tabla  
  MODIFY [COLUMN] columna1 tipo_dato [ restricciones de columna1 ],  
  MODIFY [COLUMN] columna2 tipo_dato [ restricciones de columna2 ],  
  [ ... ]  
);
```

Ej:

```
ALTER TABLE VEHICULOS  
  MODIFY COLUMN color VARCHAR(20) NOT NULL,  
  MODIFY COLUMN codTaller VARCHAR(15);
```

Alteración de tablas. Cambios estructurales.

DROP.

- ▶ **Borrar columnas:** **DROP** permite borrar columnas.

```
ALTER TABLE nombre_tabla DROP nombre_columna, DROP nombre_columna2, ...);
```

- ▶ *Ej:*

```
ALTER TABLE VEHICULOS DROP fechaMatric, DROP  
tipoFaros ;
```

Alteración de tablas. Cambios en restricciones. ADD.

Con la misma instrucción **ADD**, se pueden añadir restricciones:

```
ALTER TABLE nombre ADD CONSTRAINT nombre_restr restricción;
```

El valor de restricción, puede ser, entre otros: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK;

► *Ejemplos:* **ALTER TABLE propietario ADD CONSTRAINT veh_pk UNIQUE(DNI);**

```
CREATE TABLE marca  
(codmarca VARCHAR(15),  
descripción VARCHAR(25));
```

```
ALTER TABLE marca ADD CONSTRAINT marca_pk PRIMARY KEY(codmarca);
```

```
ALTER TABLE VEHICULOS ADD CONSTRAINT marca_fk FOREIGN KEY (marca) REFERENCES marca(codmarca);
```

Alteración de tablas. Cambios en restricciones.

DROP.

Con la misma instrucción **DROP**, se pueden borrar restricciones. Para ello debemos indicar el nombre de la restricción:

```
ALTER TABLE nombre_tabla DROP CONSTRAINT nombre_restricción;
```

► Ej:

```
ALTER TABLE VEHICULOS DROP CONSTRAINT marca_pk;
```

NOTA: no se pueden modificar CONSTRAINTS. Si se necesita cambiar una, hay que borrar la antigua (DROP) y crear una nueva (ADD).

DDL (Data Definition Language).

Creación, Modificación y Eliminación de Tablas

- ▶ No siempre es fácil llevar un orden de creación de las tablas, sobre todo si estas son numerosas. Además puede haber casos de referencias cruzadas, es decir, que cada una de las dos tablas haga referencia a la otra con una foreign key . Por eso es frecuente ver como se crean las tablas con los campos, sus tipos, y aquellas restricciones que no son de referencia a otras tablas. Una vez creadas todas las tablas, se modificarán y mediante ADD CONSTRAINT, se le añadirán todas las restricciones referenciales.
- ▶ *Ejemplo:* Vamos a crear el modelo que vimos en la unidad anterior de los coches, los clientes, los mecánicos y los trabajos realizados sobre los coches.

Son cuatro tablas:

- ▶ CLIENTE
- ▶ COCHE : necesita que esté creada la tabla CLIENTE pues hay una FK.
- ▶ MECÁNICO
- ▶ TRABAJOS: necesita que estén creadas las tablas COCHE y MECÁNICO

DDL (Data Definition Language).

Creación, Modificación y Eliminación de Tablas

► OPCIÓN 1

Creamos las tablas en orden y así podremos poner directamente las restricciones de columnaa la hora de crear las tablas. Primero MECÁNICO y CLIENTE, que no referencian a ninguna otra tabla. Después COCHE, que referencia a CLIENTE y por último TRABAJOS, que referencia a COCHE y MECÁNICO:

```
CREATE TABLE MECANICO (  
  Cod_empleado VARCHAR(5) PRIMARY KEY,  
  Nombre VARCHAR(60) NOT NULL,  
  Especialidad VARCHAR(30)  
);
```

```
CREATE TABLE CLIENTE (  
  DNI VARCHAR(10) PRIMARY KEY,  
  Nombre VARCHAR(60) NOT NULL,  
  Direccion VARCHAR(60),  
  Telefono VARCHAR(60) NOT NULL,  
  email VARCHAR(20)  
);
```

1

```
CREATE TABLE COCHE  
(Matricula VARCHAR(10) PRIMARY KEY,  
  Marca VARCHAR(30),  
  Modelo VARCHAR(30),  
  Color VARCHAR(30),  
  Dni_cliente VARCHAR(10),  
  FOREIGN KEY (Dni_cliente) REFERENCES CLIENTE(DNI)  
);
```

2

```
CREATE TABLE TRABAJOS (  
  Cod_empleado VARCHAR(5),  
  Matricula VARCHAR(10),  
  Fecha DATE,  
  Reparacion VARCHAR(30),  
  Piezas INT(100),  
  Importe DECIMAL(6,2),  
  FOREIGN KEY (Cod_empleado) REFERENCES MECANICO(Cod_empleado),  
  FOREIGN KEY (Matricula) REFERENCES COCHE(Matricula),  
  CONSTRAINT PK_Reparacion PRIMARY KEY (Cod_empleado, Matricula, Fecha)  
);
```

3

DDL (Data Definition Language).

Creación, Modificación y Eliminación de Tablas

► OPCIÓN 2

Creamos las tablas sin restricciones referenciales o de tabla y luego modificamos las tablas que necesitemos:

```
CREATE TABLE MECANICO (  
  Cod_empleado VARCHAR(5) PRIMARY KEY,  
  Nombre VARCHAR(60) NOT NULL,  
  Especialidad VARCHAR(30)  
);
```

```
CREATE TABLE CLIENTE (  
  DNI VARCHAR(10) PRIMARY KEY,  
  Nombre VARCHAR(60) NOT NULL,  
  Direccion VARCHAR(60),  
  Telefono VARCHAR(60) NOT NULL,  
  email VARCHAR(20)  
);
```

1

```
CREATE TABLE COCHE  
(Matricula VARCHAR(10) PRIMARY KEY,  
  Marca VARCHAR(30),  
  Modelo VARCHAR(30),  
  Color VARCHAR(30),  
  Dni_cliente VARCHAR(10)  
);
```

```
CREATE TABLE TRABAJOS (  
  Cod_empleado VARCHAR(5),  
  Matricula VARCHAR(10),  
  Fecha DATE,  
  Reparacion VARCHAR(30),  
  Piezas INT(100),  
  Importe DECIMAL(6,2)  
);
```

FK de COCHE a CLIENTE:

```
ALTER TABLE COCHE ADD CONSTRAINT clte_fk FOREIGN KEY (Dni_cliente)  
REFERENCES CLIENTE(DNI);
```

FKs de TRABAJOS:

```
ALTER TABLE TRABAJOS ADD CONSTRAINT mec_fk FOREIGN KEY  
(Cod_empleado) REFERENCES MECANICO(Cod_empleado);
```

```
ALTER TABLE TRABAJOS ADD CONSTRAINT coc_fk FOREIGN KEY (Matricula)  
REFERENCES COCHE(Matricula);
```

```
ALTER TABLE TRABAJOS ADD CONSTRAINT trb_pk PRIMARY KEY  
(Cod_empleado, Matricula, Fecha);
```

2

The background features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue. These shapes are primarily located on the left and right sides of the frame, creating a modern, dynamic feel.

A2: DISEÑAR LAS REGLAS DE NEGOCIO PARA EL SGBD ESPECÍFICO

Restricciones de integridad referencial

- ▶ Las actualizaciones que se realizan sobre las relaciones de la base de datos deben observar ciertas restricciones que imponen las reglas de negocio de la empresa. Algunas ya las hemos usado (UQ, NOT NULL, CHECK ...)
- ▶ Algunos SGBD proporcionan mecanismos que permiten definir estas restricciones y vigilan que no se violen. Los motores de Bases de Datos relacionales sí contemplan estas restricciones.
- ▶ Hay restricciones que imponen las reglas de negocio que se resuelven con otras funcionalidades que aportan los SGBD que veremos en capítulos posteriores. Incluso hay reglas que no se pueden resolver con los SGBD, y se incluyen en programas de aplicación
- ▶ Nos centraremos ahora en las restricciones referenciales, las de la FKs.

Restricciones de integridad referencial

- ▶ Cuando una tabla hacer referencia a otra mediante una FK, hay que indicarle qué se debe hacer, según el responsable de negocio, si altermos los datos contenidos en los campos involucrados en la relación. Para ello, a la hora de definir CONSTRAINTS de tipo FOREIGN KEY indicaremos al SGBD lo que hacer usando:
 - ON DELETE CASCADE: permite que se mantenga automáticamente la integridad referencial. Borra los valores de la clave externa correspondientes a un valor borrado de la tabla padre. Si no se pone esta opción, no se permitirá borrar valores de una tabla que sean referenciados como llave externa en otras tablas.
 - ON DELETE SET NULL: especifica que se ponga a NULL los valores de la clave externa correspondientes a un valor borrado de la tabla padre.
 - ON DELETE SET DEFAULT: si se elimina una fila de la tabla padre, todas las columnas de referencia en todas las filas coincidentes de la tabla se configuran en el valor predeterminado de la columna.
- ▶ Si no indicamos al SGBD lo que tiene que hacer, éste **no nos permitirá** borrar el registro deseado de la tabla padre.

Restricciones de integridad referencial

► Ejs: En el ejemplo del taller:

Imaginemos que queremos que cuando se elimine un cliente y, por tanto, desaparezca el NIF de la tabla, se ponga a NULL el campo Dni_cliente de la tabla COCHE. Esto sólo será posible si Dni_cliente puede ser NULL.

Además, se desea que si se cambia el DNI de un cliente, se actualice el campo Dni_cliente de ese cliente para todos los coches en los que se le haga referencia.

► Si hubiésemos conocido esto desde un principio, la FK se debería de haber definido así:

```
ALTER TABLE COCHE ADD CONSTRAINT clte_fk FOREIGN KEY (Dni_cliente) REFERENCES CLIENTE(DNI) ON DELETE SET NULL ON UPDATE CASCADE ;
```

► Si la tenemos que cambiar, borraremos con un DROP la CONSTRAINT anterior y la volvemos a generar con las restricciones incorporadas:

```
ALTER TABLE COCHE DROP CONSTRAINT clte_fk;
```

```
ALTER TABLE COCHE ADD CONSTRAINT clte_fk FOREIGN KEY (Dni_cliente) REFERENCES CLIENTE(DNI) ON DELETE SET NULL ON UPDATE CASCADE ;
```

¿Dónde puedo ver las restricciones asociadas a una tabla?

Para ver las restricciones asociadas a una tabla, tenemos **dos** opciones.

1. Entramos en el esquema donde se encuentra la tabla y hacemos la consulta:

```
SHOW CREATE TABLE nombre_tabla;
```

Obtendremos algo similar a :

```
mysql> show create table pedido;
+-----+-----+
| pedido | CREATE TABLE `pedido` (  
  `codigo_pedido` int NOT NULL,  
  `fecha_pedido` date NOT NULL,  
  `fecha_esperada` date NOT NULL,  
  `fecha_entrega` date DEFAULT NULL,  
  `estado` varchar(15) NOT NULL,  
  `comentarios` text,  
  `codigo_cliente` int NOT NULL,  
  PRIMARY KEY (`codigo_pedido`),  
  KEY `codigo_cliente` (`codigo_cliente`),  
  CONSTRAINT `pedido_ibfk_1` FOREIGN KEY (`codigo_cliente`) REFERENCES `cliente` (`codigo_cliente`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
```

¿Dónde puedo ver las restricciones asociadas a una tabla?

2. Entramos en el esquema donde se encuentra la tabla y hacemos la consulta:

```
SELECT TC.CONSTRAINT_NAME,  
       TC.TABLE_SCHEMA ,  
       TC.TABLE_NAME,  
       TC.CONSTRAINT_TYPE,  
       KCU.COLUMN_NAME,  
       KCU.CONSTRAINT_NAME,  
       KCU.REFERENCED_COLUMN_NAME,  
       KCU.REFERENCED_TABLE_NAME  
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS TC,  
     INFORMATION_SCHEMA.KEY_COLUMN_USAGE KCU  
WHERE TC.CONSTRAINT_NAME = KCU.CONSTRAINT_NAME  
AND TC.TABLE_NAME= KCU.TABLE_NAME  
AND TC.TABLE_SCHEMA= KCU.TABLE_SCHEMA  
AND TC.TABLE_SCHEMA = DATABASE()  
AND TC. TABLE_NAME='nombre_tabla';
```

¿Dónde puedo ver las restricciones asociadas a una tabla?

Obtendremos algo del tipo:

```
mysql> SELECT TC.CONSTRAINT_NAME,  
-> TC.TABLE_SCHEMA ,  
-> TC.TABLE_NAME,  
-> TC.CONSTRAINT_TYPE,  
-> KCU.COLUMN_NAME,  
-> KCU.CONSTRAINT_NAME,  
-> KCU.REFERENCED_COLUMN_NAME,  
-> KCU.REFERENCED_TABLE_NAME  
-> FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS TC, information_schema.KEY_COLUMN_USAGE KCU  
-> WHERE TC.CONSTRAINT_NAME = KCU.CONSTRAINT_NAME  
-> AND TC.TABLE_NAME= KCU.TABLE_NAME  
-> AND TC.TABLE_SCHEMA= KCU.TABLE_SCHEMA  
-> AND TC.TABLE_SCHEMA = DATABASE()  
-> AND TC.TABLE_NAME='pedido';
```

CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE	COLUMN_NAME	CONSTRAINT_NAME	REFERENCED_COLUMN_NAME	REFERENCED_TABLE_NAME
PRIMARY	jardineria	pedido	PRIMARY KEY	codigo_pedido	PRIMARY	NULL	NULL
pedido_ibfk_1	jardineria	pedido	FOREIGN KEY	codigo_cliente	pedido_ibfk_1	codigo_cliente	cliente

Preguntas

